

Introducción:

Tipos de datos:

1. Enteros (**int**): Números sin decimales.
2. Flotantes (**float**): Números con decimales.
3. Cadenas (**str**): Texto entre comillas.
4. Booleanos (**bool**): **True** o **False**.

Nota para comprobar un dato se hace de esta manera
if type(nombre) == str:

Conversión de Tipos:

1. numero_texto = "100"
2. numero_entero = int(numero_texto) # Convierte texto a entero
3. numero_decimal = float(10) # Convierte entero a 10.0

Operadores:

1. Paréntesis **()** (tienen la máxima prioridad).
2. Potencia ******.
3. Multiplicación, División, Módulo y División entera *****, **/**, **%**, **//**.
4. Suma y Resta **+**, **-**.
=
==
!=

Estructuras de control:

1. Condicional simple: **if**

Se usa para ejecutar un bloque de código **sólo si** se cumple una condición:

```
idade = 20
if idade >= 18:
    print("Es maior de idade")
```

2. Condicional doble: if-else

Para cuando tienes un "Plan B". Si la condición no se cumple, se ejecuta el bloque del else.

```
nota = 4
if nota >= 5:
    print("Aprobado")
else:
    print("Suspenso")
```

3. Condicional encadenado: if, elif, else

Cuando tienes más de dos opciones. El elif (abreviatura de else if) comprueba otra condición si la anterior falló.

```
puntos = 15
if puntos > 20:
    print("Nivel Pro")
elif puntos > 10:
    print("Nivel Medio")
else:
    print("Nivel Principiante")
```

4. Condicional múltiple: match case

```
opcion = 2
match opcion:
    case 1:
        print("Escolliches a opción 1")
    case 2:
        print("Escolliches a opción 2")
    case _:
        print("Opción non válida") # O guión baixo (_) é o caso por defecto
```

Listas y sus métodos:

1. ¿Qué es una lista?

Es una colección de elementos ordenada y mutable (puedes cambiarla después de crearla). Se definen usando corchetes []

```
# Una lista de ejemplo
frutas = ["manzana", "pera", "plátano"]
```

2. Para añadir elementos:

- **append(valor)**: Añade un elemento al **final**.

```
frutas.append("naranja") # ["manzana", "pera", "plátano", "naranja"]
```

- **insert(posicion, valor)**: Añade un elemento en el lugar exacto que quieras.

```
frutas.insert(1, "uva") # Lo mete en la posición 1, desplazando el resto
```

- Añadir sublistas a una lista existente
Puedes usar el método **append()** pasando una lista completa como argumento.

```
usuarios = [{"Ana", 25}, {"Pedro", 30}]
nuevo_usuario = ["Lucía", 22]

usuarios.append(nuevo_usuario)
# Resultado: [{"Ana", 25}, {"Pedro", 30}, {"Lucía", 22}]
```

3. Para eliminar elementos:

- **remove(valor)**: Busca el elemento y lo borra (solo la primera vez que lo encuentra)

```
frutas.remove("pera")
```

- **pop(indice)**: Saca el elemento de esa posición y te lo "da" (puedes guardarlo en una variable). Si no pones índice, saca el último.

```
ultima = frutas.pop() # Quita el último y lo guarda en 'ultima'
```

4. Para organizar y buscar:

- **sort()**: Ordena la lista (alfabéticamente o de menor a mayor).

```
numeros = [5, 2, 9, 1]
numeros.sort() # Ahora es [1, 2, 5, 9]
```

```

notas = [4, 8, 3, 10, 5]

# Ordenamos de mayor a menor
notas.sort(reverse=True)

print(notas)
# Resultado: [10, 8, 5, 4, 3]

```

- **index(valor)**: Te dice en qué posición está algo.

```
pos = frutas.index("manzana") # Devuelve 0
```

- Ordenar por un elemento específico (El truco del "Key")
Imagina que quieres ordenar por el **segundo** valor de cada sublista (por ejemplo, puntos o edad). Para eso usamos una función **lambda**.

```

# Lista de [Nombre, Puntuación]
puntuaciones = [["Ana", 45], ["Berto", 90], ["Clara", 70]]

# Queremos ordenar de mayor a menor PUNTUACIÓN (índice 1)
puntuaciones.sort(key=lambda x: x[1], reverse=True)

print(puntuaciones)
# Resultado: [['Berto', 90], ['Clara', 70], ['Ana', 45]]

```

lambda x: x[1]: Es una función rápida que le dice a Python: "Oye, para decidir quién va primero, mira solo la posición 1 de cada sublista".

reverse=True: Le da la vuelta al orden natural (que es de menor a mayor) para que el más grande quede arriba.

5. Acceso por índice:

Recuerda que en programación empezamos a contar desde **0**.

```

colores = ["rojo", "verde", "azul"]
print(colores[0]) # "rojo"
print(colores[-1]) # Truco: el -1 es siempre el último elemento

```

Trocar listas (Slicing)

Puedes sacar "pedazos" de una lista usando los dos puntos :

```
sublista = numeros[1:3] # Coge desde el índice 1 hasta el 2 (el 3 no se incluye)
```

- Cómo acceder a los datos de sublistas
Para entrar en una sublista necesitas **dos índices**: el primero para la lista grande y el segundo para el elemento de la sublista

```
menu = [ ["Café", 1.50], ["Té", 1.20], ["Zumo", 2.00]]

# ¿Cómo saco el precio del Té?
# 1. El Té está en la posición 1 de la lista grande.
# 2. El precio (1.20) está en la posición 1 de esa sublista.

precio_te = menu[1][1]
print(precio_te) # 1.20
```

Strings:

Los **strings** (o cadenas de texto) son fundamentales. En Python, un string es básicamente una secuencia de caracteres encerrada entre comillas simples (' ') o dobles (" ").

1. Para cambiar el formato:

Estos métodos son ideales para normalizar datos (por ejemplo, cuando un usuario escribe su nombre todo en minúsculas).

- `upper()`: Pone todo en MAYÚSCULAS.
- `lower()`: Pone todo en minúsculas.

```
texto = "hola mundo"
print(texto.upper())      # "HOLA MUNDO"
print(texto.title())      # "Hola Mundo"
```

2. Para limpiar y buscar:

Muy útiles cuando recibes datos de formularios o archivos.

- `strip()`: Elimina los espacios en blanco sobrantes al principio y al final.
- `replace("antiguo", "nuevo")`: Cambia un trozo de texto por otro.
- `find("texto")`: Te dice en qué posición empieza lo que buscas (devuelve -1 si no lo encuentra).
- `count("letra")`: Cuenta cuántas veces aparece algo

```
sucio = "  Python  "
print(sucio.strip())      # "Python"

frase = "Me gusta Java"
print(frase.replace("Java", "Python")) # "Me gusta Python"
```

3. Para dividir y unir:

- **split(separador)**: Rompe el string y lo convierte en una **lista**. Por defecto corta por los espacios.
- **join(lista)**: Lo contrario; une los elementos de una lista para formar un solo string.
- **slicing string[inicio:fin:paso]** :En lugar de quedarte con todo el texto, seleccionas solo la parte que te interesa.

```
# Ejemplo de split
datos = "perro,gato,loro"
lista_animales = datos.split(",") # ['perro', 'gato', 'loro']

# Ejemplo de join
nombres = ["Ana", "Berto"]
juntos = " y ".join(nombres) # "Ana y Berto"

#Ejemplo de slicing
telefono = "981-555-666"
prefijo = telefono[0:3]

if prefijo.isdigit():
    print(f"Prefijo {prefijo} válido.")
else:
    print("El prefijo contiene caracteres no válidos.")
```

4. Comprobaciones (Devuelven True o False):

- **isdigit()**: ¿Son todos números?
- **isalpha()**: ¿Son todo letras?

```
#Ejemplo de isdigit()
entrada = input("Introduce tu edad: ")
if entrada.isdigit():
    edad = int(entrada)
    print(f"Perfecto, tienes {edad} años.")
else:
    print("Error: Por favor, introduce solo números (sin letras ni espacios).")

#Ejemplo de isalpha()
nombre = input("Introduce tu nombre: ")
# Quitamos espacios antes de comprobar, porque un espacio daría False
if nombre.strip().isalpha():
    print(f"Hola {nombre}, nombre válido.")
else:
    print("Error: El nombre no puede contener números ni símbolos.")
```

5. Acceso por índice:

Al igual que las listas, puedes acceder a letras específicas usando []:

```
lenguaje = "Python"
print(lenguaje[0]) # "P"
print(lenguaje[-1]) # "n" (la última)
```

Bucles:

1. Bucle **while**:

Se usa cuando no sabes exactamente cuántas veces se va a repetir el código. Se ejecuta mientras una condición sea verdadera.

```
contador = 1
while contador <= 3:
    print(f"Vuelta número {contador}")
    contador += 1 # ¡Ojo! Si no sumas, el bucle será infinito
```

2. Bucle **for**:

Normalmente piensas en for como: "recorrer cosas".

Pero también puede funcionar como: "repetir algo cierto número de veces".

Ejecutar funciones en serie:

```
for i in range(5):
    print("Hola")
```

1. No estás recorriendo una lista real que te interese.
2. Solo usas range(5) para decir: repite 5 veces.
3. La variable i es solo un contador (0, 1, 2, 3, 4), no un elemento importante.

Dibujar patrones simples:

```
# Dibujar una pequeña pirámide
for i in range(1, 6):
    print("*" * i)
```

1. range(n): Repite el bloque n veces.
2. _: Indica que no necesitas el valor del índice, solo la repetición.
3. range(inicio, fin, salto): Te permite controlar el conteo.

Iterar:

En Python, el for es un "iterador". Sirve para recorrer los elementos de una colección (listas, tuplas, diccionarios) o un rango de números.

```
# 1. Creamos la lista
alumnos = ["Ana", "Berto", "Clara"]

# 2. El bucle recorre la lista
for alumno in alumnos:
    print(f"Hola, {alumno}")
```

1. alumnos = [...]: Definimos nuestra colección de datos. En memoria, Python sabe que hay tres elementos.
2. for alumno in alumnos::
Aquí ocurre la magia.
 - alumnos: Es la lista completa (el origen).
 - alumno: Es una variable temporal que creamos nosotros. En cada vuelta del bucle, esta variable "toma prestado" el valor del elemento que toca.
3. El "viaje" del bucle:
 - Vuelta 1: Python mira la lista, ve a "Ana", se la asigna a la variable alumno y ejecuta el print.
 - Vuelta 2: Salta al siguiente, ve a "Berto", actualiza la variable alumno y vuelve a ejecutar el print.
 - Vuelta 3: Ve a "Clara", la mete en el alumno y ejecuta el código.
4. Final: Como ya no hay más nombres en la lista, el bucle termina automáticamente. No tienes que decirle cuándo parar.

Funciones:

Una función es un **bloque de código con nombre** diseñado para realizar una tarea específica. En lugar de escribir el mismo código diez veces, lo guardas en una función y la "llamas" cuando la necesites.

- **Definición:** Usas `def`.
- **Llamada:** Usas el nombre seguido de paréntesis `()`.

1. Ámbito de las variables:

Este concepto es crucial para evitar errores. Determina dónde es "visible" una variable:

- **Variables Globales:** Declaradas fuera de las funciones. Todo el programa las ve.
- **Variables Locales:** Declaradas dentro de una función. **Solo existen ahí dentro.** Cuando la función termina, la variable desaparece.

```
x = "Soy global"

def mi_funcion():
    y = "Soy local"
    print(x) # Funciona
    print(y) # Funciona

mi_funcion()
# print(y) # ¡ERROR! 'y' no existe fuera de la función
```

2. Paso de información:

Los parámetros son variables que actúan como "puertas de entrada" para que la función reciba datos del exterior.

- **Parámetro:** El nombre en la definición (`def suma(a, b):`).
- **Argumento:** El valor real que pasas al llamar (`suma(5, 3)`)

3. Valor devuelto (**return**):

Para que una función no solo haga algo, sino que nos dé un resultado que podamos guardar en una variable, usamos la palabra clave `return`.

```
def multiplicar(a, b):  
    resultado = a * b  
    return resultado # Envía el dato hacia afuera  
  
pago_total = multiplicar(10, 5) # pago_total ahora vale 50
```

4. Recursividad:

La recursividad ocurre cuando **una función se llama a sí misma**. Es como poner dos espejos frente a frente. Es muy útil para problemas que se pueden dividir en versiones más pequeñas de sí mismos (como buscar archivos en carpetas dentro de otras carpetas).

Reglas de oro de la recursividad:

1. **Caso Base:** Una condición que detiene las llamadas (si no, el programa se cuelga).
2. **Caso Recursivo:** La llamada a la función con un dato más pequeño.

Ejemplo:

Una función recursiva es como un **eco**. Tú gritas un número, y la montaña te devuelve el número menos uno, hasta que se llega al silencio

```
def cuenta_atras(numero):  
    if numero == 0: # CASO BASE: Aquí es donde paramos  
        print("¡Ignición! 🚀")  
    else: # CASO RECURSIVO: La función se llama a sí misma  
        print(numero)  
        cuenta_atras(numero - 1) # Le pide a la función que lo haga con el siguiente número  
  
# Lanzamos el cohete  
cuenta_atras(3)
```

Objetos:

1. Propiedades y Métodos

Son los dos pilares de cualquier clase:

- **Propiedades (Atributos):** Son variables que guardan el **estado** del objeto.
 - *Ejemplo:* `color`, `velocidad_maxima`, `gasolina`.

Tipos:

- Público(nombre) = Se pueden acceder y modificar desde cualquier lugar.
- Protegidos(_nombre) = (Un guion bajo) Indica que es para uso interno o de subclases. Es solo una sugerencia.
- Privados(__nombre) = (Dos guiones bajos). Activa el *Name Mangling* (ofuscación) para dificultar el acceso desde fuera.
- **Métodos:** Son funciones que definen el **comportamiento**.
 - *Ejemplo:* `acelerar()`, `frenar()`, `repostar()`.

2. Constructor e Inicializador (`__init__`)

Es el "primer respiro" del objeto. En Python, el método `__init__` se asegura de que el objeto tenga sus propiedades listas desde el segundo uno.

3. La cláusula `self`

Es una referencia al **objeto actual**. Sin `self`, Python no sabría qué coche está acelerando si tienes diez en pantalla.

- Dentro de la clase, usas `self.propiedad` para leer o cambiar datos de **ese** objeto específico.

Ejemplo:

```
class Perro:
    def __init__(self, nombre):
        self.nombre = nombre # 'self.nombre' es la propiedad del objeto

    def ladrar(self):
        print(f"{self.nombre} dice ¡Guau!")
```

4. Método `__str__`

Define qué debe aparecer en pantalla cuando haces un `print()` de un objeto.

```
class Libro:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor

    # Definimos la representación en texto del objeto
    def __str__(self):
        return f"'{self.titulo}' escrito por {self.autor}"

mi_libro = Libro("Cien años de soledad", "García Márquez")

# Ahora print() busca automáticamente el método __str__
print(mi_libro)
# Salida: 'Cien años de soledad' escrito por García Márquez
```

5. Herencia: ¿Cuándo heredar?

Heredar es cuando una clase "hija" copia todo lo que tiene una clase "padre".

- **Regla de oro:** Usa herencia solo si puedes decir **"Es un..."**.
 - ¿Un coche **es un** vehículo? Sí -> Herencia.
 - ¿Un profesor **es una** persona? Sí -> Herencia.
 - ¿Un coche **es un** motor? **No** (el coche *tiene* un motor) -> No uses herencia.
 - **¿Cómo?** Poniendo la clase padre entre paréntesis al definir la hija.

```
from vehículo import vehiculo
class Vehiculo: # Clase Padre
    def __init__(self, marca):
        self.marca = marca

class CocheElectrico(Vehiculo): # Clase Hija
    def __init__(self, marca, bateria):
        super().__init__(marca) # super() llama al constructor del padre
        self.bateria = bateria
```

6. Setter y Getters

1. ¿Para qué sirven realmente?

1. **Validación:** El Setter puede revisar si el dato es correcto antes de guardarlo (ej: que la edad no sea negativa).
2. **Seguridad:** Puedes hacer que una propiedad sea de "solo lectura" creando el Getter pero no el Setter.

3. **Formateo:** El Getter puede devolver el dato transformado (ej: el nombre siempre en mayúsculas).

Ejemplo:

Cuenta de Usuario donde la contraseña no puede tener menos de 4 caracteres.

```
class Usuario:
    def __init__(self, nombre, clave):
        self._nombre = nombre
        # Usamos el setter incluso dentro del constructor para validar
        # la clave inicial
        self.set_clave(clave)

    # GETTER: Método para obtener el nombre de forma controlada
    def get_nombre(self):
        return self._nombre.title() # Lo devuelve con la primera en
        mayúscula

    # SETTER: Método para cambiar la clave con una regla de seguridad
    def set_clave(self, nueva_clave):
        if len(nueva_clave) >= 4:
            self._clave = nueva_clave
            print("Clave actualizada correctamente.")
        else:
            print("Error: La clave es demasiado corta (mínimo 4
            caracteres).")

# --- USO DEL OBJETO ---

user1 = Usuario("pedro", "1234")

# 1. Usamos el Getter
# No hacemos print(user1._nombre), usamos el método:
print(user1.get_nombre()) # Salida: Pedro

# 2. Usamos el Setter para intentar cambiar la clave
user1.set_clave("abc") # Salida: Error: La clave es demasiado
corta...
user1.set_clave("secret") # Salida: Clave actualizada correctamente.
```

7. Clases Abstractas

Son clases "incompletas". Sirven para definir un concepto general pero prohíben crear objetos de ellas.

- **Para qué sirven:** Para obligar a todas las clases hijas a tener los mismos métodos, pero dejando que cada una lo haga a su manera.
- *Ejemplo:* Una clase **Forma** puede ser abstracta y obligar a tener un método **dibujar()**. El **Círculo** lo dibujará redondo y el **Cuadrado** con cuatro líneas.

```
from abc import ABC, abstractmethod

# 1. Definimos la Clase Abstracta (El molde)
class MetodoPago(ABC):

    @abstractmethod
    def procesar_pago(self, cantidad):
        """Este método es obligatorio para todas las clases hijas"""
        pass

    def generar_recibo(self):
        """Un método normal que sí pueden compartir todas"""
        print("Generando recibo genérico...")

# 2. Creamos las Clases Hijas (Las que sí se pueden usar)
class Tarjeta(MetodoPago):
    def procesar_pago(self, cantidad):
        print(f"Conectando con el banco... Pago de {cantidad}€ aceptado con Tarjeta.")

class PayPal(MetodoPago):
    def procesar_pago(self, cantidad):
        print(f"Redirigiendo a PayPal... Pago de {cantidad}€ aceptado.")

# --- Uso del código ---

# pago = MetodoPago() <-- ¡ESTO DARÍA ERROR! No puedes crear un "Método de Pago" a secas.

mi_pago = Tarjeta()
mi_pago.procesar_pago(100)
mi_pago.generar_recibo()
```

8. ERROR

1. Sintaxis básica

Para lanzar un error, usas `raise` seguido del tipo de error y un mensaje entre paréntesis para explicar qué pasó.

```
def dividir(a, b):  
    if b == 0:  
        raise ZeroDivisionError("¡No puedes dividir entre cero, genio!")  
    return a / b
```

Los errores más comunes son:

- `ValueError` = El valor es incorrecto (ej: edad negativa, texto donde esperas números)
- `TypeError` = El tipo de dato no es el que querías (ej: pasas una lista donde esperas un texto).
- `IndexError` = Intentas acceder a una posición de una lista que no existe.
- `Exception` = es un error genérico el cual hacemos un nuevo `.py` para llamarlo

Exceptio ejemplo:

```
### Creamos un nuevo .py  
class DataError(Exception):  
    def __init__(self, mensaje):  
        super().__init__()  
        self.mensaje = mensaje  
    def __str__(self):  
        return "Error " + str(self.mensaje)  
  
### después lo importamos para su uso  
  
from DataError import DataError  
  
def setDia(self, dia):  
    if type(dia) == int:  
        meses_31_dias = {1, 3, 5, 7, 8, 10, 12}  
        meses_30_dias = {4, 6, 9, 11}  
  
        if self.__mes in meses_31_dias:  
            max_dia = 31  
        elif self.__mes in meses_30_dias:
```

```

        max_dia = 30
    elif self.__mes == 2:
        # febrero, revisamos bisiesto
        if (self.__ano % 4 == 0) and ((self.__ano % 100 != 0) or
(self.__ano % 400 == 0)):
            max_dia = 29
        else:
            max_dia = 28
    else:
        raise DataError(f"El mes {self.__mes} no es válido")

    if 1 <= dia <= max_dia:
        self.__dia = dia
    else:
        raise DataError(f"El día {dia} no concuerda con los días
del mes {self.__mes}")
    else:
        raise TypeError("El formato de día no es adecuado, debe ser
un entero")
    def getDia(self):
        return self.__dia

```

9. Relanzar un error (**try...except + raise**)

A veces quieres capturar un error para registrarlo en un log o avisar, pero luego quieres que el error siga su curso para que el programa principal sepa que algo falló

Ejemplo cajero:

si intentas sacar más dinero del que tienes, el programa debe "protestar"

```

class Cajero:
    # Definimos el saldo_inicial en 1000 por defecto
    def __init__(self, titular, saldo_inicial=1000):
        self.titular = titular
        self._saldo = saldo_inicial

    # --- GETTER CLÁSICO ---
    def get_saldo(self):
        return self._saldo

    # --- SETTER CLÁSICO ---
    def set_saldo(self, nueva_cantidad):
        # El Setter es el que vigila que no haya saldos negativos

```



```

        if nueva_cantidad < 0:
            raise ValueError("Operación denegada: El saldo no puede ser
negativo.")

        self._saldo = nueva_cantidad

    def sacar_dinero(self, cantidad):
        print(f"\nIntentando retirar: {cantidad}€...")
        try:
            # Calculamos cuál sería el saldo resultante
            nuevo_valor = self.get_saldo() - cantidad

            # Intentamos aplicar ese nuevo valor usando el SETTER
            self.set_saldo(nuevo_valor)

            print(f"¡Éxito! Has sacado {cantidad}€. Nuevo saldo:
{self.get_saldo()}€")

        except ValueError as e:
            # Capturamos el error que lanzó el setter para avisar al
usuario
            print(f"ERROR EN CAJERO: {e}")
            # Volvemos a lanzar el error (raise) para que el banco sepa
que falló
            raise

    def __str__(self):
        return f"Cliente: {self.titular} | Saldo: {self.get_saldo()}€"

# --- BLOQUE DE EJECUCIÓN ---
if __name__ == "__main__":
    # Al no pasarle saldo, empezará con 1000€
    cuenta = Cajero("Brais")
    print(cuenta)

    # 1. Prueba de éxito
    try:
        cuenta.sacar_dinero(200)
    except ValueError:
        print("La transacción no se pudo completar.")

    # 2. Prueba de fallo (sacar más de lo que hay)
    try:
        cuenta.sacar_dinero(1500)
    except ValueError:
        print("El sistema central ha bloqueado el sobregiro.")

```

```
print(f"\nResultado final: {cuenta}")
```

Es una excelente pregunta. ¿Por qué repetir el `try...except` en la parte final si ya lo pusimos dentro del método `sacar_dinero`?

La respuesta es por **el control total del programa**.

1. Para evitar que el programa se "rompa" (Crash)

Cuando usas `raise` dentro de una función, el error sale disparado hacia afuera. Si el programa principal no lo "atrapa" con un `try...except`, Python se detiene por completo, muestra un texto rojo gigante en la consola y deja de ejecutar el resto del código.

2. El "Efecto Dominó"

Imagina que estás procesando los pagos de **10 clientes** en un bucle.

- **Si NO usas `try...except` en la ejecución:** Cuando el Cliente 1 intente sacar más dinero del que tiene, el programa lanzará un error y **se cerrará**. Los otros 9 clientes se quedarán sin procesar.
- **Si USAS `try...except` en la ejecución:** Capturas el error del Cliente 1, avisas que falló, y el programa **sigue funcionando** para atender al Cliente 2, 3, etc.

Ficheros

Para guardar en ficheros usare un ejemplos de una lista de la compra:

Clase producto

Este archivo es común. Guárdalo con este nombre para que los demás puedan importarlo.

```
class Producto:
    def __init__(self, nombre, cantidad, precio):
        self._nombre = nombre
        self._cantidad = cantidad
        self._precio = precio

    # Getters clásicos
    def get_nombre(self):
        return self._nombre
    def get_cantidad(self):
        return self._cantidad
    def get_precio(self):
        return self._precio

    # Setters clásicos
    def set_nombre(self, nombre):
        self._nombre = nombre
    def set_cantidad(self, cantidad):
        self._cantidad = cantidad
    def set_precio(self, precio):
        self._precio = precio

    def __str__(self):
        return f"Producto: {self._nombre} | Cantidad: {self._cantidad} | Precio Unit: {self._precio}€"
```

Clase Lista:

Opción A: TXT (Texto Plano)

```
from producto import Producto

class ListaCompra:

    def __init__(self):
        self.lista_compra = []
        self.fichero = "compras.txt"

    def agregar(self, p):
        # Añade un producto a la lista de la compra
        self.lista_compra.append(p)

    def borrar(self, nombre):
        # Creamos una lista nueva vacía
        nueva_lista = []

        # Recorremos todos los productos
        for p in self.lista_compra:
            # Si el nombre es diferente, lo conservamos
            if p.get_nombre().lower() != nombre.lower():
                nueva_lista.append(p)
            # Reemplazamos la lista original por la nueva
            self.lista_compra = nueva_lista

    def ver_lista(self):
        # Devuelve la lista completa de productos
        return self.lista_compra

    def total(self):

        # Creamos una variable para acumular el total
        suma = 0
        # Recorremos todos los productos
        for p in self.lista_compra:
            # Calculamos precio * cantidad del producto
            subtotal = p.get_precio() * p.get_cantidad()
            # Lo sumamos al total
            suma += subtotal

        # Devolvemos el total final
        return suma

    def guardar(self):
```

```
# Guarda la lista en un archivo de texto
with open(self.fichero, "w") as f:

    for p in self.lista_compra:
        # Convierte cada objeto Producto en texto
        # formato: nombre,cantidad,precio
        linea =
f"{p.get_nombre()}, {p.get_cantidad()}, {p.get_precio()}\n"

        # Escribe la línea en el archivo
        f.write(linea)

def cargar(self):
    try:
        # Abre el archivo para leer los datos guardados
        with open(self.fichero, "r") as f:

            # Reinicia la lista para evitar duplicados
            self.lista_compra = []

            # Lee línea por línea
            for linea in f:

                # Quita salto de línea y separa por comas
                n, c, p = linea.strip().split(",")

                # Convierte texto → objeto Producto
                producto = Producto(n, int(c), float(p))

                # Añade el producto a la lista
                self.lista_compra.append(producto)

    except FileNotFoundError:
        # Si el archivo no existe, crea una lista vacía
        self.lista_compra = []
```

Opción B: Versión CSV (.csv)

```
import csv
from producto import Producto

class ListaCompra:

    def __init__(self):
        self.lista_compra = []
        self.fichero = "compras.csv"

    def agregar(self, p):
        # Añade un producto a la lista de la compra
        self.lista_compra.append(p)

    def borrar(self, nombre):
        # Creamos una lista nueva vacía
        nueva_lista = []

        # Recorremos todos los productos
        for p in self.lista_compra:
            # Si el nombre es diferente, lo conservamos
            if p.get_nombre().lower() != nombre.lower():
                nueva_lista.append(p)
            # Reemplazamos la lista original por la nueva
            self.lista_compra = nueva_lista

    def ver_lista(self):
        # Devuelve la lista completa de productos
        return self.lista_compra

    def total(self):

        # Creamos una variable para acumular el total
        suma = 0
        # Recorremos todos los productos
        for p in self.lista_compra:
            # Calculamos precio * cantidad del producto
            subtotal = p.get_precio() * p.get_cantidad()
            # Lo sumamos al total
            suma += subtotal
```

```
# Devolvemos el total final
return suma

def guardar(self):
    # Abre el archivo en modo escritura
    # "w" = write (borra lo anterior y escribe de nuevo)
    # newline="" evita saltos de línea extra (importante con CSV)
    with open(self.fichero, "w", newline="") as f:

        # Crea un escritor de CSV (formato tabla separado por comas)
        escritor = csv.writer(f)

        # Recorre cada producto de la lista
        for p in self.lista_compra:

            # Escribe una fila en el archivo CSV
            # Cada fila será: nombre, cantidad, precio
            escritor.writerow([
                p.get_nombre(),
                p.get_cantidad(),
                p.get_precio()
            ])

def cargar(self):
    try:
        # Abre el archivo en modo lectura
        with open(self.fichero, "r") as f:

            # Reinicia la lista para evitar duplicados
            self.lista_compra = []

            # Crea un lector CSV que separa automáticamente por comas
            lector = csv.reader(f)

            # Recorre cada fila del archivo CSV
            for fila in lector:

                # fila[0] = nombre
                # fila[1] = cantidad (texto → int)
                # fila[2] = precio (texto → float)

                producto = Producto(
                    fila[0],
                    int(fila[1]),
                    float(fila[2]))
```

```

        )

        # Añade el producto reconstruido a la lista
        self.lista_compra.append(producto)

    except FileNotFoundError:
        # Si el archivo no existe aún, crea lista vacía
        self.lista_compra = []

```

Opción C: Versión BINARIA (.bin)

```

import pickle
from producto import Producto

class ListaCompra:
    def __init__(self):
        self.lista_compra = []
        self.fichero = "compras.bin"

    def agregar(self, p):
        # Añade un producto a la lista de la compra
        self.lista_compra.append(p)

    def borrar(self, nombre):
        # Creamos una lista nueva vacía
        nueva_lista = []

        # Recorremos todos los productos
        for p in self.lista_compra:
            # Si el nombre es diferente, lo conservamos
            if p.get_nombre().lower() != nombre.lower():
                nueva_lista.append(p)
            # Reemplazamos la lista original por la nueva
            self.lista_compra = nueva_lista

    def ver_lista(self):
        # Devuelve la lista completa de productos
        return self.lista_compra

    def total(self):
        # Creamos una variable para acumular el total

```



```

        suma = 0
        # Recorremos todos los productos
        for p in self.lista_compra:
            # Calculamos precio * cantidad del producto
            subtotal = p.get_precio() * p.get_cantidad()
            # Lo sumamos al total
            suma += subtotal

        # Devolvemos el total final
        return suma

def guardar(self):
    with open(self.fichero, "wb") as f:
        pickle.dump(self.lista_compra, f)

def cargar(self):
    try:
        with open(self.fichero, "rb") as f:
            self.lista_compra = pickle.load(f)
    except (FileNotFoundError, EOFError):
        self.lista_compra = []

def guardar(self):
    # Abre el archivo en modo binario de escritura ("wb")
    # wb = write binary → se guarda en formato binario (no texto)
    with open(self.fichero, "wb") as f:

        # pickle convierte la lista de objetos Python en datos binarios
        # y los guarda en el archivo
        pickle.dump(self.lista_compra, f)

def cargar(self):
    try:
        # Abre el archivo en modo binario de lectura ("rb")
        # rb = read binary → lee datos binarios
        with open(self.fichero, "rb") as f:

            # pickle.load reconstruye los objetos originales
            # (recupera la lista de productos tal como era)
            self.lista_compra = pickle.load(f)

    except (FileNotFoundError, EOFError):
        # FileNotFoundError → el archivo no existe aún
        # EOFError → el archivo está vacío o corrupto

```

```
# Si hay error, se crea una lista vacía
self.lista_compra = []
```

Clase Menu:

```
from producto import Producto
from lista_compra import ListaCompra

def mostrar_menu():
    mi_lista = ListaCompra()

    while True:
        # Cargar datos al inicio de cada iteración del bucle
        mi_lista.cargar()

        print("\n--- APP LISTA DE COMPRA ---")
        print("1. Agregar")
        print("2. Borrar")
        print("3. Ver")
        print("4. Salir")

        opcion = input("Elige una opción: ")

        match opcion:
            case "1":
                while True:
                    nombre = input("Producto (o 'v' para volver): ")
                    if nombre.lower() == 'v': break
                    cant = int(input("Cantidad: "))
                    precio = float(input("Precio: "))

                    mi_lista.agregar(Producto(nombre, cant, precio))
                    mi_lista.guardar() # Guardamos tras agregar
                    print("Agregado.")

            case "2":
                nom = input("Nombre a borrar: ")
                mi_lista.borrar(nom)
                mi_lista.guardar() # Guardamos tras borrar
                print("Eliminado.")

            case "3":
```

```
        print("\nLISTA ACTUAL:")
        for p in mi_lista.ver_lista():
            print(p)
        print(f"TOTAL: {mi_lista.total()}€")

    case "4":
        mi_lista.guardar() # Guardar antes del break
        print("Saliendo...")
        break # Rompe el bucle while

    case _:
        print("Opción no válida.")

if __name__ == "__main__":
    mostrar_menu()
```